

TD Bases de données avancées : indexation, algorithmes de jointure, plans d'exécution

Construire un index par hachage sur les données suivantes, selon le numéro de département. On utilisera 5 pages de hachage suivant un hachage modulo, avec des pages de stockage contenant au plus 3 n-uplets.

3 Allier

36 Indre
18 Cher
9 Ariège
11 Aude
12 Aveyron
73 Savoie
55 Meuse
46 Lot
39 Jura
81 Tarn
25 Doubs
15 Cantal
51 Marne
42 Loire

Tableau d'indirection	Page 0	Page 1	Page 2	Page 3	Page 4
0,1231234	(55,Meuse)	(36,Indre)	(12,Aveyron)	(3,Allier)	(9,Ariège)
1,1334343	(25,Doubs)	(1,Aude)	(42,Loire)	(18,Cher)	(39,Jura)
2,1344545	(15,Cantal)	(46,Lot)		(73,Savoie)	
3,1348346					
4,3143434		(page de débordement)			
		(81,Tarn)			
		(51,Marne)			

2) Combien de lecture de pages faut-il effectuer pour trouver l'élément 46. On supposera que le tableau d'indirection utilise une page disque. On suppose ne pas avoir de cache mémoire.
On hache le critère de recherche modulo 5. On trouve 1. On consulte le tableau d'indirection pour trouver l'adresse de la page 1 (1334343). On charge en mémoire la page 1 et on cherche 46. On le trouve.
Bilan : 2 pages de lecture.

3) Combien de lecture de pages faut-il effectuer pour trouver l'élément 109.
On hache 109 modulo 5, qui donne 4. On charge le tableau d'indirection. On charge la page 4. On ne le trouve pas. S'il existait, il serait forcément à cet endroit. On peut conclure que 109 n'est pas dans la base.
Bilan : 2 pages de lecture.

3) Combien de lecture de pages faut-il effectuer pour trouver l'élément 101.
On hache 101 modulo 5, qui donne 1. On charge le tableau d'indirection. On charge la page 1. On ne trouve pas 101. Mais il y a une page de débordement. On la charge. On cherche dans cette page. On ne trouve pas 101. On peut conclure que 101 n'est pas dans la base.

Bilan : 3 pages de lecture.

Remarque : on voit la dégradation des performances quand il y a des débordements. S'il y avait n pages de débordement associées à la page 1, il faut charger ces n pages pour vérifier la présence d'un élément. On retrouve les mauvaises performances du stockage séquentiel.

4) Combien de lecture de pages faut-il faire pour trouver les éléments dans l'intervalle [1,7]

Il faut chercher les éléments valant 1. On charge le tableau d'indirection et on charge la page $1 \% 5 = 1$, et sa page de débordement pour trouver 1. De même pour 2, 3, 4, 5, 6, 7. Globalement, on doit accéder à toutes les pages et aux pages de débordement associées. Au minimum il faut charger $5 + 1 + 1 = 7$ pages (si on suppose ne devoir lire qu'une fois le tableau d'indirection).

Conclusion : le hachage est très efficace pour les recherches par égalité, s'il n'y a pas trop de débordement. Le hachage n'est pas efficace pour les recherches par intervalle.

Algorithmes de jointure

RQ Les données ont été modifiées par rapport au TD (suppression des doublons)

Soit les tables R et S suivantes, stockées respectivement sur 2 et 3 pages disque (chaque page peut contenir au maximum 3 n-uplets) :

R(A)	S(B)
1	8
5	5
6	3
3	4
7	7
2	6
	10
	11
	9

Soit la requête suivante : `select R.A from R,S where R.A=S.B`

- 1) Il n'y a pas d'index sur R ou S (stockage séquentiel). Donnez en algèbre la requête précédente et proposez des algorithmes possibles pour l'évaluation de cette requête.

C'est une jointure suivie d'une projection : $\Pi_A (R \text{ join}_{(A=B)} S)$

Les algorithmes du cours dans le cas où il n'y a pas d'index sont jointure par boucle imbriquée (NESTED LOOP), jointure par tri-fusion (SORT-MERGE JOIN), et jointure par hachage (HASH JOIN).

- 1) pour l'algorithme par boucle imbriquée, indiquer le nombre de lecture de page nécessaire et le nombre de comparaison de n-uplets à effectuer en mémoire vive. On supposera le SGBD incapable de stocker plus d'une page par table en mémoire vive, de même pour l'écriture du résultat.

On choisit une table directrice (par exemple R).

On charge la première page de R en mémoire vive.

On charge la première page de S en mémoire vive.

En mémoire vive, on fait les comparaisons nécessaires entre tous les n-uplets pour trouver ceux en jointure.

Résultat : 5,5

Bilan : 2 pages disques lues, et $3 \times 3 = 9$ comparaisons de n-uplets en mémoire vive.

Ensuite, on charge la deuxième page de S et on fait les comparaisons entre la première page de R et la seconde de S.

Bilan : 1 lecture de page et 9 comparaisons

On charge la troisième page de S : bilan 1 lecture de page et 9 comparaisons

On charge maintenant la seconde page de R. Il faut lui présenter toutes les pages de S.

Pour chaque page P_R de R

Charger P_R en mémoire vive

Pour chaque page P_S de S

Charger P_S en mémoire vive

Faire les comparaisons de n-uplets entre PR et PS

Bilan global : 2x3 lecture de pages de S, 2 pages de lecture de R. 8 pages lues.

Comparaisons : 2x3x9 comparaisons : 54 comparaisons

Rappel : le temps CPU est négligeable par rapport au temps de lecture disque (rapport 1 000 000).

2) Faire de même pour l'algorithme Tri-Fusion

On trie d'abord les 2 tables selon le critère de jointure (ici A et B).

Résultat

RT(A)	ST(B)
1	3
2	4
3	5
5	6
6	7
7	8
	9
	10
	11

Bilan : trier n pages disques coûte $n \log n$ lectures / écritures de pages, soit $2 \log 2 + 3 \log 3 = 1+2 = 3$, mais il faut au moins lire toutes les pages, donc au minimum 5.

En CPU, pour trier n valeurs, c'est la même formule mais pour le nombre de n-uplets, soit $6 \log 6 + 9 \log 9$

On fusionne ensuite les listes triées :

On charge la première page de RT et la première page de ST

On initialise les pointeurs en début de page. On vérifie l'égalité des n-uplets et on avance le côté inférieur à l'autre.

Il consomme la première page de RT, on charge la seconde.

On consomme la seconde page de RT, on charge la troisième.

Quand il n'y a plus de n-uplets dans une des tables, on peut s'arrêter.

Bilan : on a lu toutes les pages de RT et 2 pages de ST (on avance tout le temps).

Bilan : $2+2=4$ pages de lecture.

Bilan global : 5 (tri) + 4 (fusion) : 9 pages

Comparaison CPU : quand il n'y a pas de doublons, on est sûr d'avancer un pointeur sur l'une ou l'autre des tables. A la fin d'une table l'algorithme s'arrête. Dans les pire des cas on fait le nombre de n-uplets global en comparaison : ici au max 15 comparaisons.

Rq. Si les données étaient déjà triées, la jointure aurait coûté : 4 lecture de page

Bilan : boucle imbriquée : 8 pages

Tri-fusion : 9 pages (4 dans le bon cas)